

Izmjene u implementaciji u odnosu na prvobitnu specifikaciju

Projekat: ezZkvi - Aplikacija za provjeru znanja i pripremu putem kvizova (Grupa 7, Tim 1)

U toku implementacije sistema došlo je do određenih odstupanja i proširenja u odnosu na prvobitnu specifikaciju i početni dizajn (dijagram klasa, MVC dijagram klasa i ERD). Sve izmjene su nastale kao rezultat praktičnih zahtjeva tokom razvoja, korištenja standardnih biblioteka ASP.NET Core okruženja, potrebe za boljom organizacijom koda i boljim korisničkim iskustvom.

U nastavku su navedene najznačajnije izmjene i pojašnjenja u odnosu na prvobitnu projektnu dokumentaciju.

1. Korištenje ASP.NET Core Identity okvira umjesto vlastite klase Korisnik

Prvobitni dijagram klasa predviđao je apstraktnu klasu Korisnik sa vlastitim atributima (ID, Username, Email, Lozinka) i metodama za prijavu, odjavu i registraciju. U implementaciji je model Korisnik naslijedio klasu IdentityUser iz ASP.NET Core Identity okvira, čime su autentifikacija, heširanje lozinki, potvrda e-mail adrese i upravljanje ulogama (Admin, Moderator, Student) prepušteni standardnom ASP.NET Core Identity rješenju.

Umjesto da se ručno implementira potpuno odvojena tabela korisnika, korisnički model je vezan za IdentityUser iAspNetUsers tabelu. Upravo je to i zahtijevao osmi projektni zadatak, "Implementacija korisničkog modela".

2. Odobravanje korisničkih naloga i potpuno brisanje korisnika

Specifikacija je predviđala jednostavnu registraciju i prijavu korisnika. U implementaciji je ovaj proces proširen dvostepenom verifikacijom: nakon registracije korisnik prvo potvrđuje svoju e-mail adresu, a zatim njegov nalog mora odobriti administrator.

U tu svrhu je u modelu Korisnik dodano polje IsApproved (migracija AddUserApproval), a administratorski panel je dobio pregled svih korisnika sa akcijama odobravanja, odbijanja i oduzimanja pristupa. Korisnik čiji nalog nije odobren ne može se prijaviti u sistem.

Pored toga, brisanje korisnika je implementirano kao transakciono kaskadno brisanje kroz zaseban servis UserDeletionService. Zajedno sa korisničkim nalogom brišu se njegove kviz sesije, stavke sesija, statistika i poslani feedback, čime se sprječava ostajanje nepovezanih zapisa u bazi podataka.

3. Praćenje aktivnosti korisnika (LastActivity)

U modelu Korisnik je naknadno dodano polje LastActivity (migracija AddLastActivity), koje se automatski ažurira kroz middleware pri zahtjevima prijavljenog korisnika.

Administrator u pregledu korisnika vidi da je korisnik trenutno aktivan ako je od njegove zadnje aktivnosti prošlo najviše 5 minuta. Dakle, aktivnost se ne određuje stalnom konekcijom korisnika, nego poređenjem vremena zadnje evidentirane aktivnosti sa pragom od 5 minuta. Radi smanjenja

nepotrebnih upisa u bazu, vrijednost LastActivity se ne mora ažurirati pri svakom zahtjevu ako je već nedavno ažurirana.

Ova funkcionalnost nije postojala u prvobitnoj specifikaciji ni u dijagramima, ali je dodana radi boljeg administratorskog pregleda stanja korisnika.

4. Rezultat kviza pohranjen u klasi KvizSesija umjesto zasebne klase RezultatKviza

Prvobitni dizajn, posebno interfejs IKviz u dijagramu klasa, predviđao je zasebnu klasu RezultatKviza koju vraćaju metode završiKviz() i dohvatiRezultat(). U implementaciji je odlučeno da se rezultat pohranjuje direktno u entitet KvizSesija kroz polja BrojTacnih, Procenat, DatumPocetka i DatumZavrsetka (migracija AddKvizSesijaRezultat).

KvizSesija se koristi za čuvanje osnovnih podataka o započetom kvizu, vremenu početka i završetka, statusu i konačnom rezultatu. Posebno je važan entitet KvizSesijaPitanje, jer u njemu ostaju pitanja koja su dodijeljena konkretnoj sesiji i odgovori koje je student već odabrao ili preskočio. Zbog toga se student može vratiti na već započeti kviz ako tokom rada izađe iz njega, sve dok ne istekne tajmer.

Nakon završetka kviza, odnosno nakon isteka vremenskog ograničenja, pojedinačne stavke sesije mogu se obrisati iz baze, dok agregirani rezultat ostaje sačuvan u KvizSesija i StudentStatistika. Time se zadržavaju podaci potrebni za rezultat i leaderboard, a ne čuvaju se nepotrebni detalji završene sesije.

Tokom implementacije je također odlučeno da simulaciju kviza radi samo student. Prvobitni dizajn je predviđao da i moderator ima metode za pokretanje i završavanje kviza, ali je u finalnoj verziji to izostavljeno jer moderator upravlja sadržajem, dok je rješavanje kvizova studentska funkcionalnost. Zbog toga interfejs IKviz nije realizovan kao zaseban interfejs, a logika rada kviza implementirana je u StudentControlleru i povezanim servisima.

5. Tok simulacije: nastavak prekinute sesije, preskakanje i zaključavanje pitanja

Implementacija je proširena mogućnošću nastavka prekinute simulacije kviza. Odgovori studenta se tokom simulacije kontinuirano pohranjuju u entitet KvizSesijaPitanje, kroz polje OdgovorId (migracija AddActiveQuizSessionProgress), pa student može nastaviti sesiju koja je u toku, npr. nakon zatvaranja preglednika.

Uvedena je i mogućnost preskakanja pitanja. Preskočeno pitanje se interno označava posebnom vrijednošću, a jednom odgovoreno ili preskočeno pitanje se zaključava, tako da student naknadno ne može mijenjati odgovor.

U skladu s tim, enumeracija StatusSesije je izmijenjena: umjesto prvobitnih vrijednosti AKTIVAN / ZAVRSEN / U_TOKU, implementirane su vrijednosti U_TOKU / ZAVRSEN / ISTEKAO. Status ISTEKAO označava sesiju kojoj je isteklo vremensko ograničenje.

6. Konkretizovan sistem bodovanja sa negativnim poenima

Specifikacija je samo načelno navodila da se tačni odgovori boduju i da se rezultat računa po unaprijed definisanoj formuli. U implementaciji je ta formula konkretizovana sistemom bodovanja sa

negativnim poenima: tačan odgovor nosi +0,1 poena, netačan odgovor -0,03 poena, a preskočeno ili neodgovoreno pitanje 0 poena.

Ukupan broj osvojenih poena ne može biti negativan, jer je ograničen na minimalno 0. Procenat uspjehnosti računa se kao odnos osvojenih i maksimalno mogućih poena sesije.

Na ekranu rezultata student vidi osvojene i maksimalne poene, kao i pregled tačnih, netačnih i preskočenih pitanja.

7. Parametri kviza i obavezna ABCD struktura pitanja

Parametri simulacije su tokom razvoja konkretizovani: kviz sada sadrži 5 pitanja uz vremensko ograničenje od 3 minute. Algoritam nasumičnog odabira i ponderisanja po težini iz specifikacije realizovan je kvotama: 1 teško, 2 srednja i 2 laka pitanja, uz nasumično miješanje.

Ako u nekoj kategoriji nema dovoljno pitanja, preostala mjesta se popunjavaju nasumičnim pitanjima iz ostalih kategorija, bez dupliranja. Na taj način se poštuje planirana struktura kviza kada je to moguće, ali sistem i dalje može formirati kviz ako baza nema dovoljan broj pitanja za svaku težinu.

Dodatno je pooštren kriterij validnosti pitanja: u kviz ulaze samo ABCD pitanja sa tačno 4 ponuđena odgovora i tačno jednim tačnim odgovorom. Redoslijed pitanja i ponuđenih odgovora se nasumično miješa.

8. Nova klasa StudentStatistika i izmijenjen leaderboard

U model je naknadno uvedena klasa/tabela StudentStatistika (migracija AddStudentStatistika), koja kumulativno čuva statistiku studenta po predmetu: broj odigranih kvizova, ukupan broj pitanja i broj tačnih odgovora. Statistika se ažurira pri svakom završetku kviza.

Leaderboard se više ne računa prolaskom kroz sve pojedinačne kviz sesije, nego direktno iz tabele StudentStatistika, što je efikasnije i jednostavnije za prikaz rang-liste.

Specifikacija je predviđala asinhronu obradu u pozadini uz periodično ažuriranje. U implementaciji nije uveden zaseban pozadinski servis, nego se rang-lista izračunava kada se otvori ili osvježi prikaz leaderboarda. Stranica tada preko posebne akcije Leaderboard/Data dohvaća samo podatke potrebne za tabelu, pa se prikaz može osvježiti bez ponovnog učitavanja cijele stranice.

Omogućeno je i filtriranje leadboarda po predmetu, što početnom specifikacijom nije bilo posebno predviđeno.

9. Proširenje entiteta Feedback (autor i predmet)

Prvobitni model predviđao je da Feedback sadrži status, tip, datum slanja i sadržaj. U implementaciji su dodana polja KorisnikId, koje označava korisnika koji je poslao feedback, i opciono PredmetId, koje označava predmet na koji se prijedlog ili prijava greške odnosi (migracija FeedbackAutorIPredmet).

Time moderator i administratori pri pregledu prijava imaju potpuniji kontekst. U pregledu su dodane i statistike, poput broja neobrađenih prijava, broja prijedloga pitanja i broja prijava grešaka.

10. Validacija, kaskadno brisanje i reorganizacija kontrolera za sadržaj i feedback

Naknadno su uvedeni pomoćni servisi za rad sa sadržajem koji nisu postojali u početnom dizajnu. ContentValidation normalizuje unos, uklanja suvišne razmake, provjerava da nazivi predmeta i oblasti sadrže samo dozvoljene znakove (slova, brojevi i razmaci), te sprječava unos duplikata naziva.

ContentDeletionService omogućava sigurno kaskadno brisanje. Brisanjem predmeta ili oblasti brišu se povezana pitanja, odgovori, sesije i feedback, čime se čuva konzistentnost baze.

Pregled i upravljanje sadržajem reorganizovani su u tabove za pitanja, oblasti i predmete, sa zasebnom pretragom i filterima po svakom tabu. Dodatno je logika za sadržaj izdvojena u zaseban ContentController, dok je logika za prijedloge pitanja i prijave grešaka izdvojena u zaseban FeedbackController.

U ranijoj implementaciji veći dio ove logike bio je smješten u ModeratorController, što nije bilo dovoljno logično jer sadržajem i feedbackom ne upravlja samo moderator, nego i administrator. Zasebni kontroleri bolje prate MVC strukturu aplikacije i stvarne odgovornosti dijelova sistema.

11. Tehničke i infrastrukturne izmjene

Za pohranu podataka korišten je PostgreSQL uz Npgsql provajder za Entity Framework Core, a projekat uključuje Docker podršku kroz Dockerfile i docker-compose konfiguraciju za jednostavnije pokretanje aplikacije.

Pri pokretanju aplikacije automatski se izvršavaju migracije baze, kreiraju se osnovne uloge i testni nalozi (admin, moderator, student). Uvedena je i globalna autorizaciona politika, tako da sve stranice zahtijevaju prijavu korisnika, uz prilagođene stranice za neprijavljene korisnike, zabranjen pristup i HTTP greške.

Korisnička sesija koristi klizno produženje i ističe nakon perioda neaktivnosti, što predstavlja dodatnu sigurnosnu mjeru koja nije bila posebno navedena u specifikaciji.